CSC490A2 - Group 8

Isaac Abell - 1008831329, Thomas(Yehyun) Lee - 1008992217,
Zeling (Zoey) Zhang - 1007714779, Jessica Zhang- 1008494130

Table of Contents

Part One: Aspirational Datasets....1
1.1 Paired Reference....1
1.2 Audio Sync Ground Truth Dataset....2
1.3 Choreography Segmentation Ground Truth....2
1.4 Performance Quality Dataset....3
1.5 Expert Feedback Corpus....3
1.6 Technique classification (Out-of-scope for current MVP)....3
Part Two: Reality Check (Useful for us)....4
2.1 Public Dance Datasets....4
2.2 Public Audio Datasets....8
2.3 Non-public datasets we can scrape....9
2.4 Non-public datasets we can generate....10
Part Three: Data-processing pipelines....12
3.1 Dancer alignment validation Data Generation Pipeline....12
3.2 Audio Alignment Algorithm Validation Data Generation Pipeline....16
3.3 Audio Alignment + Segmentation (EBS) Pipeline....20
3.4 Dance similarity score Algorithm validation Data Generation Pipeline....25
3.5 User Input Cleaning Pipeline....27
Part Four: Infrastructure as Code Implementation....29
4.1 Main Objective....29
4.2 IaC framework and project structure....29
4.3 Multiple Environments....32
4.4 Deployment Workflow....32
4.5 Validation evidence from run logs....33
4.6 Maintainability....33
4.7 Known limitations and next improvements....33
Part Five: Disaster Recovery Demonstration....34
5.1 What we simulated....34
5.2 What we verified in the demo....34
5.3 Demo Video....34
5.4 Disaster recovery result....35
Final Notes....37
Works Cited....38

Part One: Aspirational Datasets

Our product goal is to **help a learner improve** their dance techniques by aligning the user's dance video and reference video -> segmenting into **learnable units** -> scoring each unit -> generating actionable feedback. The data we *wish* we had maps directly to these four steps.

Each step demands a different kind of dataset: alignment needs paired recordings and timing ground truth; segmentation needs boundary annotations that reflect “learnable chunks”; scoring needs quality labels that separate style-level execution from camera/body differences; and feedback needs expert-written comments that connect what the model detects to what a dancer should practice. The datasets below represent what we would want if we had a “magic wand.”

1.1 Paired Reference

To reliably align and compare two performances, we would ideally have a dataset that mirrors the *exact* app setting: a reference dance video and multiple user recordings of the same choreography.

What we’d want it to contain

- Many pairs of dancers are performing the same choreography to the same song
- The key label is precise temporal alignment: start/end of shared subsections, which are truly comparable.
- Recording variability: distance to camera, angle, lighting, partial occlusion, room acoustics
- Explicitly label the dance genre/style

Why it matters

It directly evaluates the alignment module and distribution shift (e.g., hip-hop vs ballet) and enables level-conditioned feedback (beginner vs advanced precision cues). It also gives us controlled slices to evaluate scoring fairness across body types and camera setups once alignment is correct.


```
[
  {
    "pair_id": "choreo_017__ref_vs_user_004",
    "choreo_id": "choreo_017",
    "song_id": "song_017",
    "ref_video_uri": "s3://bucket/ref/choreo_017.mp4",
    "user_video_uri": "s3://bucket/user/choreo_017_user004.mp4",
    "genre": "hip-hop",
    "expertise": "beginner",
    "alignment_gt": {
      "ref_start_sec": 3.20,
      "ref_end_sec": 63.20,
      "user_start_sec": 5.87,
      "user_end_sec": 65.87,
      "shared_len_sec": 60.0,
      "time_warp": [
        {"ref_t": 3.20, "user_t": 5.87},
        {"ref_t": 33.20, "user_t": 35.95},
        {"ref_t": 63.20, "user_t": 65.87}
      ]
    },
    "capture_meta": {
      "device": "iPhone",
      "fps": 30,
      "resolution": "1080p",
      "snr_db": 10.5,
      "view": "front"
    }
  }
]
```

1.2 Audio Sync Ground Truth Dataset

Even if two videos are of the same choreography, the learner's recording often contains intro/outro chatter, environmental noise, or drift. A reliable pipeline needs a canonical musical structure so that feedback like "late on beat 6" is meaningful. A dedicated audio dataset would provide labels for beat times, downbeats, tempo changes, and phrase structure (e.g., 8-beat/16-beat groupings).

What we'd want it to contain

- Songs representative of typical dance tempos
- Ground truth beat timestamps (beats and downbeats) and tempo curves
- Optional phrase-level segmentation aligned to musical structure

```
[
  {
    "song_id": "song_017",
    "audio_uri": "s3://bucket/audio/song_017.wav",
    "sample_rate": 22050,
    "beats_sec": [0.41, 0.93, 1.44, 1.95],
    "downbeats_sec": [0.41, 3.71, 7.02],
    "tempo_bpm_curve": [
      {"t_sec": 0.0, "bpm": 148.2},
      {"t_sec": 30.0, "bpm": 149.0}
    ],
    "phrases": [
      {"start_beat": 1, "end_beat": 32, "label": "verse"},
      {"start_beat": 33, "end_beat": 64, "label": "chorus"}
    ]
  }
]
```

Why it matters

This dataset supports two critical features:

1. **Audio-based synchronization** (align user recording to a reference track robustly)
2. **Musically meaningful segmentation** (segments that respect rhythm rather than arbitrary time windows)

1.3 Choreography Segmentation Ground Truth

Once alignment is solved, the next question is what constitutes a "learnable unit." Fixed 8-beat cuts are a good starting point, but they often cut through moves. The ideal dataset includes movement-aware choreography boundaries—where a sequence of moves forms a coherent chunk that a learner can practice.

What we'd want it to contain

- Boundary timestamps for choreo groups
- Multi-annotator boundaries (because "best segmentation" is subjective)
- Optional boundary reasons (e.g., "pattern change," "combo ends")

```
[
  {
    "video_id": "choreo_017_ref",
    "video_uri": "s3://bucket/ref/choreo_017.mp4",
    "annotator_id": "coach_03",
    "boundaries_sec": [6.2, 11.8, 17.6, 25.1],
    "labels": ["intro ends", "arm combo ends",
    "footwork switch", "turn sequence begins"],
    "agreement_score": 0.78
  }
]
```

Why it matters

This dataset enables segmentation that minimizes cutting between moves and supports user-controlled granularity ("bigger chunks vs smaller chunks") while staying musically grounded.

1.4 Performance Quality Dataset

What we'd want it to contain

- Same choreo performed by many dancers
- Ratings per segment (not just per full video)
- Rubric dimensions (timing, posture, balance, isolation, power/control, cleanliness)

Why it matters
Provide multiple performances of the same choreo, with quality ratings and *structured rubrics* that separate timing issues from posture and technique issues.

```
[
  {
    "performance_id": "choreo_017_user004",
    "choreo_id": "choreo_017",
    "segment_id": "seg_03",
    "t_start_sec": 12.4,
    "t_end_sec": 16.1,
    "rubric_scores": {
      "timing": 2,
      "posture": 4,
      "balance": 3,
      "isolation": 2,
      "power_control": 3
    },
    "overall_score": 0.62,
    "rater_id": "coach_01"
  }
]
```

1.5 Expert Feedback Corpus

In addition to segment-level text feedback, an ideal feedback dataset would include fine-grained visual annotations (keyframes, still images, or short clips) linked to coaching commentary. This supports supervision for evidence selection: identifying which moment best justifies a critique and which frame should be surfaced to the learner. The unit of annotation is preferably move-level (or micro-technique level), rather than whole-video summaries, to avoid diluting actionable guidance.

```
[
  {
    "video_id": "vid_001_rock",
    "video_uri": "s3://.../vid_001_rock.mp4",
    "comment_id": "c001",
    "comment_start_sec": 2.0,
    "comment_end_sec": 12.0,
    "comment_level": "move",
    "comment": "..."
  },
  {
    "img_id": "img_001_rock",
    "img_uri": "s3://.../img_001_rock.png",
    "comment_id": "c001",
    "comment_level": "pose_snapshot",
    "comment": "..."
  }
]
```

1.6 Technique classification (Out-of-scope for current MVP)

A further aspirational dataset would annotate **named techniques** (e.g., locking elements, footwork primitives) with clip-level labels to support the retrieval of targeted tutorials and drill libraries. We treat this as out of scope for the current course deliverable because it requires a curated technique ontology and substantial expert labeling; however, we note it as a plausible extension path.

```
[
  {
    "clip_id": "lock_001",
    "clip_uri": "s3://.../lock_001.mp4",
    "technique": "uncle_sam_points"
  }
]
```

Part Two: Reality Check (Useful for us)

We acknowledge that there are high-quality **public human motion datasets**, such as Human3.6M (Ionescu et al., 2014), that provide standardized pose annotations and are widely used for benchmarking pose estimation and human motion modeling. However, these datasets are largely captured in controlled lab settings and are dominated by generic daily activities rather than dance-specific movement vocabularies. As a result, they provide limited coverage of choreographic structure, stylistic execution, and music-conditioned timing cues, which are central to our application. Since our MVP relies on extracting 2D motion signals via pose/"motion capture" pipelines, the primary bottleneck is not obtaining any pose data, but obtaining pose data that reflects the distributional properties of dance (rapid limb changes, self-occlusion, turns, style-dependent expressiveness). Therefore, we prioritize specialized dance datasets for evaluation and downstream adaptation, using general-purpose motion datasets only as auxiliary resources for debugging pose extraction or stress-testing basic tracking behavior rather than as core datasets for fine-tuning dance-specific model behavior.

2.1 Public Dance Datasets

Relevant Item	Description							
	Choreo	Samples per Choreo	Subject	Genre	Camera Views	Expertise Level	Segmentation Boundary	Seconds
AIST++ (Li et al., 2021) & AIST Dance DB (Tsuchida et al., 2019)	101	10	30	10	9	❌	❌	18,694
ADVSUC (Endo et al., 2024)	-	-	-	10	1	❌	✅	18,694
ImperialDance (Zhong et al., 2023, Zhong & Demiris, 2024)	20	100	5	-	-	✅	❌	69,300
FineDance (Li et al., 2023)	346	-	27	22	-	❌	❌	52,560

Relevant Item	Commentary

AIST++: https://google.github.io/aistplusplus_dataset/	Per frame, AIST++ includes:
	● 9-view camera intrinsics/extrinsics
	● 17 COCO-format joints in 2D
	● 24 SMPL(Skinned Multi-Person Linear model) pose parameters: how each body part is rotated; global scale and translation (where the body is located in 3D space)
	
	Strength
	● Multi-view is gold. It lets us test invariance to camera placement/angle without needing real users yet.
	● Strong for training representation/embedding or alignment because we can generate positives/negatives (same dancer, different view = positive; different dancers/moves = negative).
	● Broad genre coverage is helpful for generalization and ensuring scoring isn't overfit to one movement type.
	  ○ Old School (Break, Pop, Lock and Waack) and New School (Middle Hip-hop, LA-style Hip-hop, House, Krump, Street Jazz and Ballet Jazz).
	  ○ "85% of basic choreographies and 15% of advanced choreographies" (Li et al., 2021).
	Limitations:
	
	● AIST++ is recorded in studio lighting with professional cameras rather than on phones, which will likely be the main format of user videos; in practice, user recordings will involve occlusions, off-frame limbs, motion blur, and lighting/audio noise.
	● No "quality" labels: low pose distance ≠ better dancing; it only measures similarity, not coaching value.
	● No native clip-level coaching segmentation; we may need to create clip windows or use external annotations. Although segmentation boundaries are not natively part of AIST++, segmentation annotations exist from other work (Endo et al., 2024) built on AIST videos.
	
	How we'd use it:
	● **Primary MVP use (controlled validation):** Use AIST++ as a "lab environment" to generate *scoring validation pairs* (reference vs synthetically transformed "student") for our Dancer Alignment Validation Data Generation Pipeline (scale/rotate/translate/aspect-ratio + time drift + negative sampling). This lets us verify scoring invariance to recording setup while holding choreography constant.
	
	● **Secondary use (debug-only):** Use multi-view + calibration and SMPL as an analysis/debug signal for failure cases (e.g., view-specific pose extraction errors), not a runtime dependency for the MVP.

AIST: https://aistdanc	AIST is the earlier version/baseline dataset in the same ecosystem as AIST++. The key question for us is: do we need the "extra structure" AIST++ provides (especially **multi-view + camera calibration + richer pose signals**) to make our evaluation

edb.ongaaccel.jp/	meaningful? ● AIST offers dance video/motion data, but **with less complete multi-view + calibration detail** than AIST++. ● **AIST++ >= AIST** in our project due to calibrated multi-view analysis, consistency tests, and controlled positive/negative sampling. **AIST** may still help for raw volume, but it doesn't give the same "robustness knobs." **Strength** ● Can still provide additional dance coverage and data volume. ● Useful as supplemental pretraining material if it's easier to load/convert. **Limitations:** ● **Harder to do robustness experiments** without the strong multi-view + calibrated setup. ● Less ideal for our core need: proving the scoring is stable under viewpoint/recording differences. **How we'd use it:** ● It's likely that we will use both **AIST and AIST++ as the primary** dataset for scoring/alignment validation and synthetic student generation.

ADVSUC (Automatic Dance Video Segmentation for Understanding Choreography): https://github.com/rendob/dance-segmentation	A manually annotated **dance-segmentation-point** dataset built on videos from the **AIST Dance Video Database**. The goal is to mark boundaries where one movement phrase ends and the next begins—i.e., "clip-level segmentation" that lets an app jump to meaningful chunks rather than arbitrary time windows. Per video, it contains:
	● Segmentation points for 1410 videos in AIST Dance DB.
	● Labels are provided in two coordinate systems:
	  ○ segmentation points in counts (musical beats)
	  ○ segmentation points in seconds
	● Split into basic choreography and advanced choreography
	
	Strength
	● **Gives us phrase-level boundaries**, which is exactly what AIST++ lacks natively. This turns "a full routine" into coachable units (segments) so our scorer can operate per phrase instead of arbitrary sliding windows.
	
	● Aligns naturally with beat structure via "count" labels, which will be useful if we want timing-aware feedback ("you're late on the hit at count 8") without needing perfect audio onset detection in the MVP.
	
	● Enables cleaner evaluation: we can report score/alignment quality **per segment** and aggregate, instead of one noisy whole-video score.
	
	● It's also directly connected to a practical segmentation model recipe (keypoints + mel-spectrogram → TCN → boundary peaks), which gives us a future path to auto-segment user uploads without hand-labeling.
	
	Limitations:
	● Only provides **where to cut**, not **what is wrong**. There are no coaching labels like "arm angle incorrect" or "timing issue," so it supports *localizing feedback* but not generating the feedback content by itself.

	● It does not provide boundaries for unseen user videos; it provides supervision to build/validate a segmenter. ● Annotation granularity is boundary points (events), not dense per-frame phase labels—good for segmentation, but less directly usable if we later want fine-grained action-state labeling. **How we'd use it:** ● Primary MVP use: use segmentation points to define **standard phrase clips** and run our pose-based alignment/scoring per phrase (pose extraction → segment slicing → alignment → segment score → aggregated routine score). ● Use it as "ground truth boundaries" to validate any lightweight auto-segmentation heuristic we build (e.g., motion-energy peaks) and to support a future feature that lets users loop a phrase and focus practice on the exact segment that needs work.
ImperialDance: https://github.com/YunZhongNikki/ImperialDance-Dataset	**Strength** ● **Expertise level labelling** is extremely helpful, making it a good sanity check for whether our scorer correlates with expertise. ● **Longitudinal progress signal:** it includes **~100 recordings per dancer per choreography**, which naturally encodes *practice over time*. Additionally, repeated choreos help isolate performance quality vs choreography differences. **Limitations:** ● Mainly, 3D body position data, and our MVP first focused on 2D extraction. 3D to 2D representation conversion may be non-trivial depending on the skeleton format and representation. ● Labels don't tell us *what* is wrong, only experience "level," which limits feedback generation. ● If it's not raw video, it won't test our full upload → pose extraction stack. **How we'd use it:** ● The longitudinal progress signal advantage is directly relevant to a future feature where we track a user's progress across repeated attempts and summarize their **recurring personal errors** (e.g., "you're consistently late on this hit" or "your left arm collapses on this phrase").n convert it cleanly into our 2D pose representation. ● Not MVP dependency**:** used only for offline correlation sanity checks if conversion is easy (skill label vs score), otherwise conceptual grounding.
FineDance: https://github.com/li-ronghui/FineDance	Released data is organized into**:** ● motion/: SMPL-H motion files (hand-aware body model) ● music_wav/: music audio in .wav ● music_npy/: pre-extracted music features (following the AIST++ feature style) ● label_json/: metadata including song name and genre labels

	Strength ● More fine-grained motion representations than the other datasets. Strong signal for **hand motion**, which is a real differentiator for style-level evaluation. ● Useful for research-backed justification that "2D joints alone may miss important quality cues," shaping our roadmap. **Limitations:** ● **Preprocessing overhead** and skeleton mismatch risk (SMPL-H vs COCO-17). ● It's mainly represented in 3D motion parameterizations, which doesn't match our MVP's "2D-from-RGB" constraint; it could distract from shipping if treated as primary. ● Still lacks direct coaching annotations tied to user mistakes. **How we'd use it:** ● Treat as **supplementary / future work**: use it to inform what *we're missing* with COCO-17, and potentially as a later-stage dataset if we build adapters (SMPL-H → 2D joints or augment our pose extractor). For MVP, it's not a core dependency.

2.2 Public Audio Datasets

Relevant Item	Description	Commentary
GTZAN: https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification (Tzanetakis & Cook, 2002)	One thousand 30s tracks evenly distributed across 10 genres, with 100 tracks per genre**.** Per track, it includes: ● Audio format (TFDS version): **22050 Hz**, **mono**, **16-bit**, **.wav**. ● **pre-extracted features** in addition to audio.	**Strength** ● Fixed-length **30-second** clips give a clean, repeatable unit test for **alignment** and **feature-matching** logic when fully aligned long-form dance-video datasets are not available. ● Broad genre coverage reduces the risk that alignment only works for one spectral "style" of audio (e.g., only pop/EDM-like beats). ● Standardized format makes experiments easy to reproduce across teammates and runs. **Limitations** ● GTZAN is a genre dataset, not an alignment dataset—there is **no ground-truth "match" label** between different clips, so evaluation must be via constructed tests. ● Audio is relatively clean compared to real-world phone capture; we need explicit noise stress tests to approximate realistic interference. **How we'd use it** ● Use each 30-second clip as a base signal to validate alignment on controlled transformations (time-shift, tempo perturbation, cropping/offset). ● Treat "success on 30-second clips" as validation of the core feature-matching mechanism; longer durations mainly

		increase the amount of overlapping signal (more anchors) rather than changing the fundamental alignment logic. • Pair with DEMAND to systematically evaluate robustness under realistic background noise at varying SNR.
DEMAND: https://zenodo.org/records/1227121 (Thieman et al., 2013)	One thousand 30s tracks evenly distributed across 10 genres like Rock, Jazz, and Classical with real-world environmental noise recordings from diverse indoor/outdoor settings, designed for testing robustness under realistic noise. Per recording, DEMAND includes: • Recordings captured with a **16-channel microphone array** (multichannel noise). • Commonly distributed as **.wav**, **48 kHz**, **16-bit** audio (per dataset metadata listings). • Public distribution under **CC BY-SA 3.0** is widely referenced in dataset catalogs.	**Strength** • Provides **continuous noise** recordings, which avoids the artificial periodicity you get when looping short noise samples—better for realism in robustness tests. • Diverse environments let us test multiple interference profiles (e.g., broadband rumble vs. mid-frequency babble), which is useful for diagnosing failure modes of alignment features. • Multichannel capture gives the dataset high-fidelity noise structure (even if we downmix to mono for MVP). **Limitations** • It's **noise-only** (no music), so it's not directly useful for alignment by itself—its value is as a corruption source. • Environmental categories don't automatically match "phone recording" noise distributions; we still need to choose mixing strategies and SNRs that reflect our expected user conditions. **How we'd use it** • Overlay DEMAND noise onto GTZAN clips to generate realistic degraded audio without looping artifacts, then measure how alignment performance changes as we vary SNR. • Use different environments (e.g., café-like babble vs. street rumble) to create targeted masking regimes and stress-test robustness against high-interference acoustic settings.

2.3 Non-public datasets we can scrape

Relevant Item	Description	Commentary
Scraped Reference–Cover Video Pairs (e.g., Instagram/TikTok	A scraped dataset of (reference choreography video, user cover video) pairs specifically where both use the same song/choreography. Metadata includes URL, upload date, resolution, and optional	**Strength** • Closest naturally-occurring approximation of the product use case at scale: reference + learner cover recordings with real-world variation. • Enables diversity in dancer body types, recording conditions, backgrounds, and camera angles—useful for generalization.

“Dance Cover” Ecosystem)	hashtags/keywords to cluster by choreography.	**Limitations** ● Licensing/terms-of-service constraints: scraped videos often cannot be redistributed, and usage may be restricted to research/internal evaluation. ● Pairing quality is noisy: “same song” does not guarantee same choreography; hashtags can be misleading. **How we’d use it** ● Use for internal evaluation and stress-testing only (not redistributed). ● Use lightweight filters: same audio fingerprint + duration overlap + pose similarity threshold to reduce pairing noise.
Scraped Step-by-step Tutorial Videos	A dataset of tutorial videos where instructors explicitly teach choreography in chunks (e.g., “counts 1–8”, “next 8-count”). These videos often contain spoken segment boundaries that can act as weak supervision for “learnable unit” segmentation.	**Strength** ● Provides a practical definition of “learnable chunk” grounded in instruction, not purely beat structure. ● Can yield weak boundary labels via transcript cues (“from count 1”, “repeat this part”). **Limitations** ● Weak/noisy supervision: instructors vary widely in chunking strategy and terminology. ● Transcription errors and ambiguous phrasing require robust filtering. **How we’d use it** ● Use as exploratory data for future segmentation improvements beyond fixed 8-beat windows. ● Compare tutorial chunk boundaries with ADVSUC boundaries for sanity checks.

2.4 Non-public datasets we can generate

Relevant Item	Description	Commentary

Synthetic Scoring/Alignment Validation Dataset	A synthetic validation dataset generated by transforming a reference video to create a "student" variant with controlled spatial (scale/rotate/translate/aspect ratio) and temporal (offset/drift) perturbations, plus negative samples (different choreography). This corresponds directly to your existing "Dancer alignment validation Data Generation Pipeline."	**Strength** ● Provides deterministic expected behaviors: invariance to camera geometry perturbations; sensitivity to timing offsets and wrong choreography. ● Helps isolate scoring/alignment logic from the variability introduced by real human performance differences **Limitations** ● Synthetic perturbations do not capture all real-world variation (e.g., biological execution errors, balance adjustments). ● If used alone, it may overestimate robustness. **How we'd use it** ● Use as a primary unit test and regression suite for alignment/scoring changes. ● Use alongside the "Phone Reality Set" to avoid overfitting to synthetic artifacts.
LLM-Synthesized Feedback Annotations	A dataset of pseudo-coaching comments generated by a role-played LLM prompted with measurable signals (e.g., beat offset statistics, joint angle deviations, stability metrics) rather than raw video alone. Each generated comment is stored alongside the numeric evidence used to produce it.	**Strength** ● Scales the feedback layer when expert annotation is unavailable, enabling fast prototyping of feedback UX and evaluation of content templates. ● By binding text to evidence keys, we can enforce "grounded" generation constraints (the model is only allowed to reference metrics we compute). **Limitations** ● Not a substitute for expert coaching; it may encode generic advice or incorrect priorities. ● Requires careful prompt constraints and auditing to prevent hallucinated claims. **How we'd use it** ● MVP use: generate consistent, template-like feedback to validate product flow and user comprehension. ● Evaluation: treat as weak labels; validate against a small human-reviewed subset.

Part Three: Data-processing pipelines

Our project evaluates dance performance by aligning a user's recording to a reference dance and producing a similarity/quality score that is robust to real-world recording noise (camera angle, distance, phone mic quality, background interference, etc.). To make this system reliable and testable, we designed our data processing workflow around two complementary layers:

- **Production ingestion layer:** raw videos and audio (from public datasets today, and from user uploads post-MVP).
- **Validation + transformation layer:** controlled test-case generation, feature normalization, and segment-level artifacts that make scoring reproducible and debuggable.

This section presents **five data-processing pipelines** (implemented or scaffolded) that collectively cover ingestion, cleaning, transformation, and our data lake design for reproducible evaluation in a team setting.

Pipelines (5)

1. **Video Alignment Validation Data Generation Pipeline**
2. **Audio Alignment Algorithm Validation Data Generation Pipeline**
3. **Audio Alignment + Segmentation (EBS) Pipeline**
4. **Dance Similarity Score Algorithm Validation Data Generation Pipeline**
5. **User Input Cleaning + Canonicalization Pipeline (post-MVP / post-sandbox)**

For each pipeline, we include:

- **Data schemas** (JSON artifacts and key fields)
- **Pipeline diagram + technologies used**
- **When the pipeline runs / use cases** (generation vs validation vs runtime)
- **Code for an initial version** (GitHub file path / entrypoint; implemented or scaffolded)
- **Next steps** (features not implemented yet)

3.1 Dancer alignment validation Data Generation Pipeline

The Dancer alignment pipeline creates validation datasets by applying mathematical affine transformations (such as scaling, rotation, and aspect ratio changes) to reference dance videos. This ensures the algorithm evaluates the user's kinematic performance rather than their recording environment, guaranteeing that scores are unaffected by camera placement, distance from the lens, or video resolution differences.

When will the pipelines run and for which use cases

- Data Generation: this runs when we are generating the validation dataset. It should ideally only run once and is simply used to create a dataset to validate our scoring model works correctly
- Model Validation: This pipeline will be run every time we create or modify our scoring algorithm in order to test it and ensure it performs as expected.

Data Processing Steps

Step 1: Core Motion Selection Source:

- Source: A random 3–5 second clip (T) is sampled from a high-quality dance dataset (AIST++).
- Constraint: The clip must contain high-variance motion (i.e., not standing still) to ensure the algorithm has active movement to score.

Step 2: Parallel Transformation:

- The reference clip is duplicated to create a "Student" clip (Clip S), which is processed via 2D affine transformations to simulate specific recording variances:
- Spatial Simulation: We apply randomized matrix operations including Scaling (0.8x–1.2x to simulate distance), Translation (±10% to simulate off-center positioning), and Rotation (±15◦ to simulate crooked camera angles).
- Morphological Simulation: We alter the aspect ratio (width stretch 0.9x–1.1x) to act as a proxy for validating robustness against different body builds and camera driver distortions.
- Constraint: All transformations must ensure the dancer remains fully within the frame boundaries to prevent MediaPipe body detection failures.

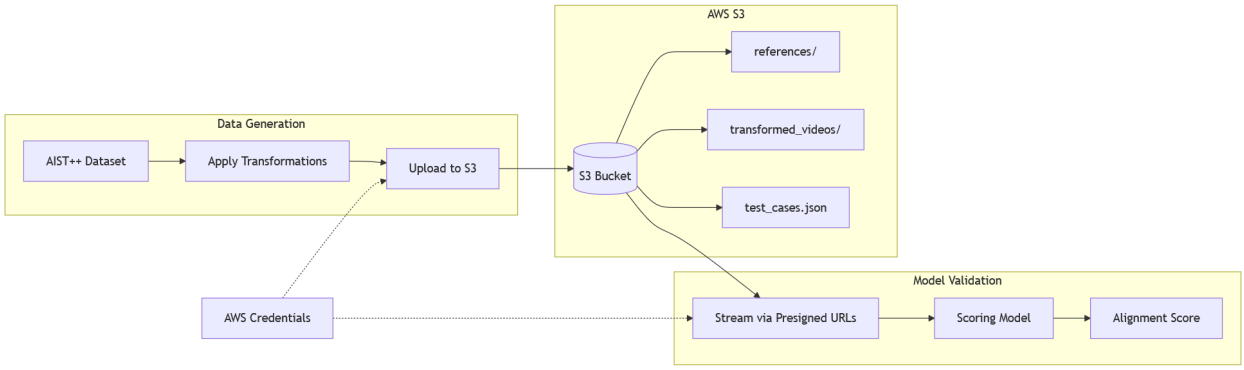
Step 3: Temporal & Distractor Injection:

- To validate temporal sensitivity and negative matching, we generate specific "failure" cases:
- Drift Injection: We trim the start of the Student clip by +0.1s to +0.5s. The algorithm must detect this offset and lower the score accordingly.
- Negative Sampling: A completely distinct clip is selected from the dataset (Clip D). The algorithm must score the pair (T vs. D) near 0%.

Step 4: Ground Truth Export

- For every generated pair, a JSON artifact is created containing the S3 keys for the Reference and Student clips, the transformation type applied, and the expected score range.

Pipeline Diagram



Reference videos are assigned a sequential ID (R001, R002, …). Each transformed video's ID prepends its parent reference ID followed by a transform index (R001_T001, R001_T002, …) followed by its transformation type, making the relationship between videos and the transformation immediately clear.

Example	File Path
Reference # 1	references/R001_dance_reference.mp4
Reference #1 Scale 0.8×	transformed_videos/R001_T001_dance_spatial_scale_0.8.mp4
Reference #1 Rotation 10°	transformed_videos/R001_T002_dance_spatial_rotation_10.mp4

Data Schemas

```
[
  {
    "test_id": "scale_test",
    "ref_s3_key": "references/reference.mp4",
    "student_s3_key": "students/student_scale_0.8.mp4",
    "transformation_type": "spatial_scale",
    "param_value": 0.8,
    "expected_score_min": 0.95, // High score expected despite scaling
    "expected_score_max": 1.0
  },
  {
    "test_id": "offset_test",
    "ref_s3_key": "references/reference.mp4",
    "student_s3_key": "students/student_offset_0.5s.mp4",
    "transformation_type": "offset",
    "param_value": 0.5,
    "expected_score_min": 0.40, // Low score expected due to timing error
    "expected_score_max": 0.60
  }
]
```

Storage Strategy

Decision:

- The validation dataset will be hosted in Cloud Object Storage (AWS S3) rather than locally.

Justification:

- Single Source of Truth: S3 provides a centralized, immutable repository. We ensure that all team members and CI runners validate the model against the exact same version of the generated data.
- Decoupling: Separation of Generation from Validation light compute, run frequently. We do not want to force every developer to install FFmpeg and generate 5GB of video just to run a test.

Code (initial version)

- A2/pipelines/video data processing

Technologies Used

- OpenCV – Video reading/writing, frame transformations (scale, rotate, translate)
- NumPy – Array math for transformation matrices
- MediaPipe – Body pose detection to keep dancer on-screen
- Boto3 – AWS S3 uploads and presigned URL streaming
- Click – Command-line interface
- AWS S3 – Cloud video storage
- AWS STS – Credential validation

Decisions and Trade-offs

Decision 1: Synthetic 2D Transformations vs. Real "Bad" Dancers

- **Decision:** We utilize mathematical 2D affine transformations (rotation matrices, scaling) on the reference video rather than recording real humans performing errors.
- **Rationale:** Precision. We can mathematically guarantee that Clip B is exactly "Clip A rotated 15 degrees." This isolates the variable being tested. If we used real videos, a low score could be due to timing, limb length, or balance, making root-cause analysis impossible.
- **Trade-off:** Synthetic transformations lack biological physics. A "stretched" video does not perfectly mimic a different body type, and a "time-shifted" video does not capture the loss of balance a human experiences when missing a beat.

Decision 2: Aspect Ratio Distortion vs. 3D Body Simulation

- **Decision:** We simulate different body types (heights/weights) by altering the video aspect ratio (stretching/squashing) rather than using complex 3D mesh simulations.

- **Rationale:** This serves as a sufficient proxy to verify that the pose normalization layer (which should mathematically standardize height and width) is functioning correctly. It avoids the complexity of generating synthetic 3D humans.
- **Trade-off:** Visual realism. The video will look unnaturally distorted to a human observer, even though the skeleton data remains valid for the algorithm.

Decision 3: Multiple Transformation Stress Test

- **Decision:** We include a subset of test cases where all transformations (Scale + Rotate + Translate + Aspect Ratio) are applied simultaneously.
- **Rationale:** Unit tests catch isolated issues from one type of transformation, but Combination transformation tests ensure the normalization pipeline doesn't suffer from compounding errors (e.g., a rotation making a translation calculation incorrect).
- **Trade-off:** Debugging complexity. If a "Combo" test fails, it is difficult to determine which specific transformation caused the failure without reverting to the isolated test cases.

Decision 4: Streaming Ingestion (Presigned URLs) vs. Bulk Download

- **Decision:** The validation worker streams video data directly from S3 using Presigned URLs, rather than downloading the dataset to disk.
- **Rationale:** Latency and startup time. Downloading the full dataset (gigabytes of videos) would make testing slow to start. Streaming allows the scoring engine to process frames immediately as they arrive.
- **Trade-off:** Network Jitter. If the S3 connection lags, the video decoder might drop frames or stall. We must implement "Blocking Read" logic in the loader to ensure the scoring algorithm doesn't penalize the dance for what is actually a network issue.

Next steps

Although this pipeline successfully generates data and allows users to test models locally, we still need to write the infrastructure code to automatically run these tests in AWS after changes to the model are pushed to our main branch in github so that we can automatically test before deploying any changes to production.

3.2 Audio Alignment Algorithm Validation Data Generation Pipeline.

The Audio Alignment pipeline generates validation datasets by degrading audio clips with environmental noise and frequency filters while maintaining precise timestamp records. Its purpose is to verify that the algorithm can accurately synchronize a user's recording to a reference track based solely on audio content, ensuring robustness against variations in microphone quality or background interference.

When will the pipelines run and for which use cases

- Data Generation: This runs when we are generating the validation dataset. It should ideally only run once, and is simply used to create a dataset to validate our scoring model works correctly
- Model Validation: This pipeline will be run every time we create or modify our alignment algorithm (assuming it continues to exclusively use audio) in order to test it and ensure it performs as expected.

Data Processing Steps

Step 1: Core Signal Selection

- Source: A random track (T) is sampled from the GTZAN dataset.
- Constraint: The entire clip is treated as the "Match Window" to maximize feature density.

Step 2: Parallel Degradation

The audio track is duplicated to make two clips (Clip A and Clip B), which are processed independently to ensure spectral divergence:

- Acoustic Simulation (EQ): Each branch applies a randomized Bandpass Filter (e.g., Clip A: 200Hz–3kHz; Clip B: 100Hz–5kHz) to simulate different microphone frequency responses.
- Environmental Injection: randomly selected audio clips from DEMAND are mixed into both clips at randomized Signal-to-Noise Ratios (SNR) ranging from 5dB to 20dB.

Step 3: Temporal Shifting

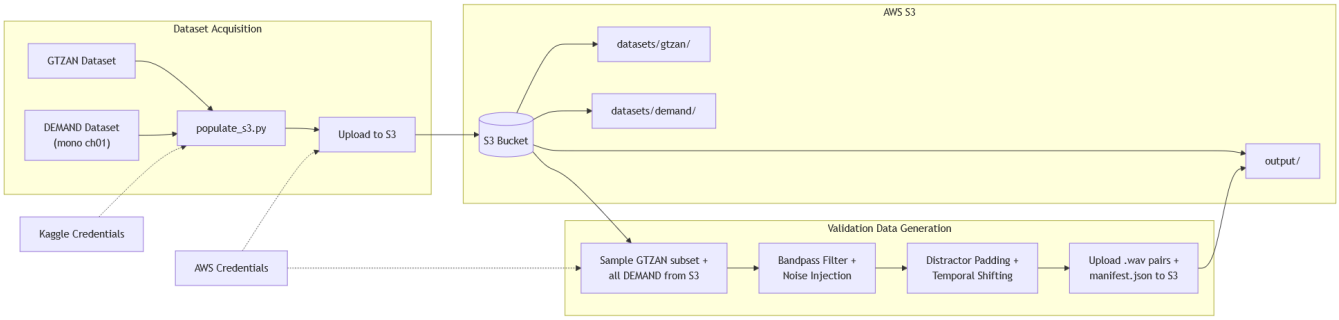
To generate the target offsets, we append "Distractor" audio to the start and end of both clips.

- Method: Random segments are sampled from the source dataset (distinct from T) and concatenated to the start/end of the degraded clips.
- Result: The algorithm must distinguish the target audio from semantically similar "distractor" audio, rather than simply detecting the onset of noise vs. silence.

Step 4: Ground Truth Export

For every generated pair, a JSON artifact is created containing the precise sample indices where T begins and ends in both files, providing a deterministic answer key for testing.

Pipeline Diagram



Data Schemas

```
[
  {
    "test_id": "val_001_rock",
    "clip_1_path": "./data/val_001_A.wav",
    "clip_2_path": "./data/val_001_B.wav",
    "clip_1_start_sec": 2.0, // The ts where the shared audio BEGINS in clip 1
    "clip_1_end_sec": 12.0, // The ts wehre the shared audio ENDS in clip 1
    "clip_2_start_sec": 5.5, // The ts where the shared audio BEGINS in clip 2
    "clip_2_end_sec": 15.5 // The ts where the shared audio ENDS in clip 2
  }
]
```

Storage Strategy

Production (Source of Truth): AWS S3

- **Role:** S3 acts as the canonical "Source of Truth" for all raw and generated datasets.
- **Structure:**
  - s3://[bucket-name]/datasets/raw/: Immutable copies of GTZAN and DEMAND.
  - s3://[bucket-name]/datasets/validation/v[x.x]: Versioned outputs of generated validation pairs.
- **Justification:** S3 ensures that every engineer and the CI/CD pipeline runs tests against the exact same binary files, eliminating "it works on my machine" inconsistencies caused by local file corruption or version mismatches.

Development (Rapid Iteration): Local Cache / Generation

- **Role:** Engineers can run the pipeline locally for rapid debugging.
- **Workflow:** Developers have two options:
  1. **Download:** Pull a specific versioned dataset from S3 to their local machine for testing.

2. **Generate:** Run the generation scripts locally (bypassing S3 upload) to test algorithm changes instantly without spending time pulling data or running the model in AWS.

Code (initial version)

- A2/pipelines/Audio data processing

Technologies Used

- Librosa – Audio loading, resampling to mono at target sample rate
- SciPy – Butterworth bandpass filter design and application
- SoundFile – Writing processed audio clips to WAV format
- NumPy – Audio signal manipulation, RMS calculations, noise mixing
- Boto3 – AWS S3 uploads and presigned URL streaming
- AWS S3 – Cloud video storage
- AWS STS – Credential validation

Decisions and Trade-offs

Decision 1: Semantic "Distractor" Padding vs. Silence/Noise

- **Decision**: We utilize random music/speech clips from the dataset as padding rather than silence or Gaussian noise.
- **Rationale:** This prevents the algorithm from cheating by detecting simple energy onsets (i.e., finding where silence stops). It forces the model to perform true content matching, distinguishing the target signal from other valid audio signals.
- **Trade-off:** If the algorithm fails, it is harder to diagnose *why*. With silence padding, failure is obvious (it didn't find the signal). With semantic padding, a failure could mean the algorithm correctly found a "similar" spectral feature in the distractor audio (a False Positive), making root-cause analysis significantly more time-consuming.

Decision 2: Environmental Noise vs. Synthetic (White) Noise

- **Decision:** We inject real-world environmental recordings (non-stationary noise) rather than mathematically generated White/Pink noise.
- **Rationale:** Synthetic noise is stationary (constant mean/variance) and easily removed by standard filters. Real-world noise (e.g., a car horn, a distinct voice) has dynamic time-frequency features that overlap with the target, providing the only valid test for real-world robustness.
- **Trade-off:** Unlike Gaussian noise which is generated via one line of code (np.random.normal), this approach requires curating, cleaning, and normalizing a secondary "Noise Dataset," adding dependency management and build complexity to the pipeline.

Decision 3: Double-Blind Degradation

- **Decision:** Both files in the pair are degraded; neither is a "clean" reference.

- **Rationale:** This mirrors peer-to-peer alignment scenarios (e.g., syncing two user videos) where no "master" copy exists. It validates that the algorithm will work for the actual audio clips that are uploaded.
- **Trade-off:** When both signals are heavily degraded, the "true" mathematical peak of cross-correlation becomes wider and flatter. This inherently limits the maximum theoretical time-resolution we can validate (e.g., we may only be able to certify alignment to within ±50ms rather than ±1ms), as the "exact" match point becomes acoustically ambiguous.

Decision 4: Cloud Storage (S3) vs. Local Storage

- **Decision:** We will host the master datasets and generation pipeline on AWS S3, transitioning away from pure local storage.
- **Rationale:** As the team and project scale, sharing 1.7GB+ datasets via Google Drive or git-LFS becomes unmanageable. S3 provides a centralized, versioned, and programmatic way to manage data. It allows us to trigger validation jobs automatically in the cloud (e.g., via AWS Batch or Lambda) without requiring an engineer's laptop to be running.
- **Trade-off:** This introduces network latency and AWS costs (storage + egress). However, the cost is negligible for this data size (<5GB), and the consistency benefits outweigh the latency.

Decision 5: Hybrid Development Workflow (Local Caching)

- **Decision:** While S3 is the master, we will maintain tooling to run the pipeline entirely locally or cache S3 data locally.
- **Rationale:** Requiring a cloud upload/download for every single code change slows down the "write code -> test" loop. By allowing the generator to write to a local `./output` directory during development, we reduce the amount of time we will have to spend waiting when testing our models for run, enabling faster iteration.
- **Trade-off:** This requires maintaining two "modes" in our scripts (e.g., a `--local` flag vs. a `--s3` flag), which adds slight code complexity to the file I/O handlers.

Next steps

Although this pipeline successfully generates data and allows users to test models locally, we still need to write the infrastructure code to automatically run these tests in AWS after changes to the model are pushed to our main branch in github so that we can automatically test before deploying any changes to production.

3.3 Audio Alignment + Segmentation (EBS) Pipeline

This pipeline aligns two video recordings to a **shared audio timeline** and then segments the aligned overlap into **coachable motion units** for downstream scoring. In our implementation,

segmentation is embedded inside the audio alignment pipeline because alignment establishes the shared time axis required to map segment boundaries consistently across both clips.

Downstream scoring is computed **per segment** and aggregated, so segment boundary quality directly affects feedback usefulness.

When will the pipelines run and for which use cases

- **Validation generation:** run when creating audio validation pairs
- **During scoring:** run every time user uploads a pair (reference + user) to align timelines and produce segment boundaries
- **During development:** run frequently when tuning alignment/segmentation heuristics

Data Processing Steps

Step 1: Audio Extraction + Canonicalization

With two input videos (Reference + User), extract audio tracks via FFmpeg and standardize to a common format (mono, 22,050 Hz) for stable feature computation.
Output: Two canonical WAV signals ready for matching.

Step 2: Auto-Alignment (Shared Window Discovery)

Compute onset-strength envelopes for both clips and estimate the best temporal offset via cross-correlation to find where the shared audio content overlaps across the two clips.
Output: A shared overlap window expressed as timestamps in each clip (clip_1_start/end, clip_2_start/end, shared_len_sec).

Step 3: Beat Grid Estimation on the Shared Window (Reference-side)

Run beat tracking on the reference clip's shared window to estimate tempo and beat timestamps.
Output: beats_shared_sec (beat times relative to the shared window start) + estimated BPM.

Step 4: Beat Coverage + Confidence Gate

Extend the beat grid to cover the full shared window (extrapolation near boundaries), then apply a confidence gate (minimum beat count + tempo bounds + regularity).
Output: Either (A) beat grid marked as reliable, or (B) trigger fallback segmentation.

Step 5: Phrase-Aligned Segmentation (8-beat) + Fallback

If confidence passes:

- Identify the 8-count downbeat phase on the beat grid and segment the shared window into 8-beat phrases (drop incomplete tail).

If confidence fails:

- Fallback to fixed-time segmentation (3.0s chunks) across the shared window.
  Output: Segment boundaries on the shared timeline.

Step 6: Boundary Projection + Artifact Export

Map each shared-window segment boundary back to absolute timestamps in both clips and export a single JSON artifact.
Output: ebs_segments.json containing alignment, beat metadata, segmentation mode, and per-clip segment timestamps.

<u>In summary, up until the A2 submission for segmentation pipeline, we implemented:</u>

1. **Auto-alignment** using onset-envelope cross-correlation to find the best temporal offset between the two clips.
2. **Beat-informed segmentation** when confidence is high:
   - estimate beats on the reference shared window
   - infer the 8-count downbeat phase
   - slice into **8-beat segments** aligned to musical phrasing

3. **Fallback segmentation** when confidence is low: fixed-time chunking into **3.0-second windows**

Observed failure mode/ Decision & Trade-offs

We built a quick local testing UI to validate the alignment and segmentation algorithms. As a result, with the eight-beat segmentation rule, we still get a lot of cutting choreography mid-move:

- Moves split across boundaries lose motion context
- Transition information is lost (important for musicality and control)

This motivated our continued research into more robust phrase boundary detection

Why dance segmentation differs from music-only segmentation

We realized choreography learning and curation are not solely beat-locked or fixed-window; they often occur in **motion blocks/phrases of variable length**, even within a single song, let alone across songs. Beat grids help, but segmentation quality depends on musicality + motion structure.

Two segmentation directions (research-backed)

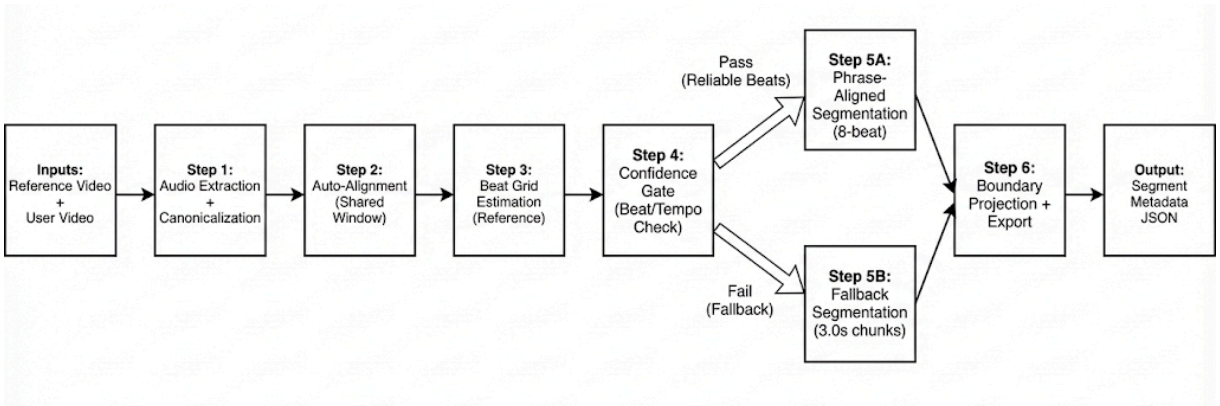
<u>Approach 1 — Rule-based segmentation (hard-coded rules)</u>

A pragmatic baseline is to apply segmentation rules using audio + motion signals (e.g., motion energy changes, pauses/holds, boundary heuristics) (Okada, 2015).

Approach 2 — Learned segmentation model

Train a boundary predictor using labeled segmentation points (Endo et al., 2024) with the ADVSUC dataset, which includes segmentation points on the AIST Dance DB.

Diagram



Data schema

```
[
  {
    "alignment": {
        "clip_1_start_sec": float,
        "clip_1_end_sec": float,
        "clip_2_start_sec": float,
        "clip_2_end_sec": float,
        "shared_len_sec": float,
    },
    "beat_tracking": {
        "estimated_bpm": float,
        "num_beats": int, // total (detected + extrapolated)
        "num_beats_detected": int, // librosa-detected only
        "confidence": {
            "median_interval_sec": float,
            "coefficient_of_variation": float,
            "passed": bool,
        },
        "downbeat_phase": int | None,    //0-7, None if fallback
        "source": "librosa.beat.beat_track",
    },
    "beats_shared_sec": list[float], // All beat times relative to shared window
    "segmentation_mode": str, // "eight_beat" | "fixed_time"
    "segments": list[Segment], // Segment includes seg_id, beat_idx_range
  }
]
```

Technologies Used

- **FFmpeg** – Extract audio tracks from input videos and convert to a canonical WAV format.
- **Librosa** – Load audio at a fixed sample rate (22,050 Hz), compute onset-strength envelopes, and perform beat tracking to estimate BPM and beat timestamps.

- **NumPy** – Numerical operations on feature arrays (normalization, beat-grid extrapolation, timestamp conversions) and general signal/array manipulation.
- **SciPy (signal)** – FFT-based cross-correlation (via convolution) to estimate the temporal lag between two clips during auto-alignment.
- **Python JSON I/O** – Serialize the final alignment + beat + segment boundaries into ebs_segments.json for downstream scoring.
- **AWS S3 + Boto3** – Persist versioned JSON artifacts in object storage for reproducible evaluation and CI runs.

Storage Strategy

Decision: We store the pipeline's **JSON artifacts** (alignment + beats + segments) as small, versioned files in **AWS S3** for reproducibility, while using **local caching** for repeated iteration during development.

We design the layout as:s3://<bucket>/datasets/derived/ebs/v1/<pair_id>/ebs_segments.json

What we store

- **Always persisted (S3):** ebs_segments.json (single canonical artifact for this pipeline)
- **Cached locally:** extracted WAV and intermediate feature arrays (onset_env, etc.) to avoid recomputation during tuning

Justification

- **Single source of truth**: downstream scoring only needs ebs_segments.json, so storing it centrally prevents "works on my machine" drift.
- **Decoupling**: scoring can be tested repeatedly without re-running alignment/beat tracking.
- **Speed**: feature extraction is the bottleneck; local caching makes the dev loop fast.

Code (initial version)

In feat/ebs-pipeline branch: A2/pipelines/Audio data processing/ebs_segment.py

Next steps

- Implement rule-based segmentation baseline (Scitepress 2015)
- Train learned boundary model using ADVSUC (Endo et al., 2024)
- Add segment-quality metrics (boundary stability + move-cut rate)
- Add tempo-aware segmentation so segment length is beat-defined when reliable
- Enable storage strategy in S3

3.4 Dance similarity score Algorithm validation Data Generation Pipeline

We anticipate the user input video to be often different from the source video since the user is likely to be learning from someone else. This then requires us to develop a dance similarity scoring algorithm that is resistant to noise like body types, background and more, and only evaluate similarity based on the execution of moves. The Dance Scoring Algorithm validation Data aims to establish data to create a scoring that is consistent and reliable for evaluating user input to the source video. We would use this dataset to tune our overlay mechanism and scoring.

When will the pipelines run and for which use cases

The pipeline only runs once (or multiple times <5) to generate the dataset to support the creation of a good user-source similarity algorithm that is used for almost all use cases involving the evaluation of user performance.

Data Processing Steps

Our strategy is to gain a dataset with alignment of execution with difference in dancer. Thus, we filter through the datasets in part2 for clips of different dancers performing the same dance at the same level of execution. We train the model to produce similar dance scores for these clips.

Step 1: Find dances with multiple dancers at the same level

- Source: filter through all the relevant datasets() in part2 [this ended up to just be AIST] to obtain a dataset of videos of different dancers doing the same dance. We choose to group by dance because the level and style does not matter.
  Filtered source folder structure:
  Dance
  - Dancer1
    - Vid1
    - Vid2
    - … (if applicable)
  - Dancer2

Step 2: Preprocessing

- *Optional alignment using The Dancer alignment pipeline if needed
- Segmentation: For each pair, use **FFmpeg** to cut into 3–5 second clips
  - We work under the assumption that the filtered dance videos are of the same length, if not, we perform a cut off at the shortest video to maintain the same length.
- Pairing: generate combinations of dancers under the same dance (ex: dance1-001-003).
  Preprocessed clips folder structure:
  Dance

Dancer1

clip1 (each 3-5 sec)

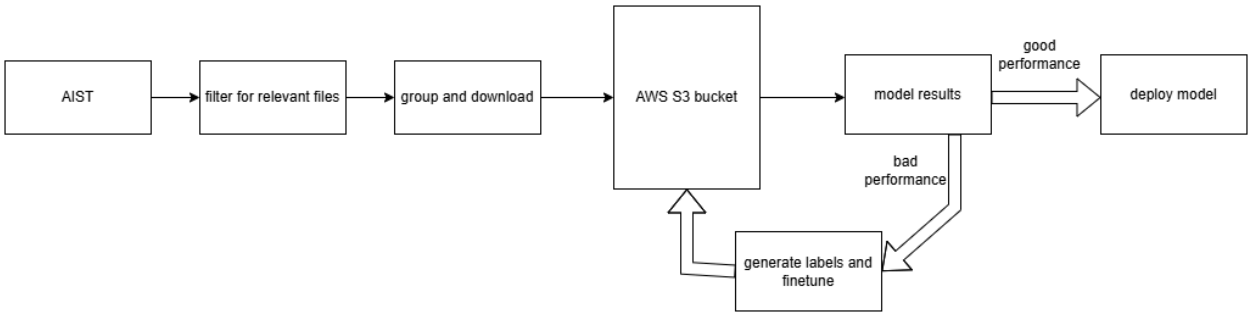
clip2

…

Dancer2

…

Pipeline Diagram



Data Schema

```
[
  {
    "dance_id": "sourceDatasetName_001",
    “dancer_id1”: “001”,
    “dancer_id2”: “002”, //clip path follows dance_dancerid_clipid format
    "clip_1_path": "./data/dace_id/dancerid1_001.wav",
    "clip_2_path": "./data/dace_id/dancerid2_001.wav",
    “clip_id”: “001”,
    "clip_start_sec": 2.0, // The timestamp where the dance BEGINS in clip
    "clip_end_sec": 5.0, // The timestamp where the dance ENDS in clip
  }
]
```

Storage Strategy

Decision:

- The validation dataset will be hosted locally on the development machine and in cloud object storage (e.g., AWS S3).

Justification:

1. **Scale**: The resulting dataset after preprocessing and cleaning would be small enough to be stored on one computer. We also upload to AWS S3 for engineering best practices.
2. **Latency**: Training and validation loops require high-frequency file I/O (opening thousands of small audio clips). Local storage will make running these loops significantly faster than S3 HTTP requests.
3. **Reusability**: We only plan to use this for training the and the scoring algorithm and the dataset will not be reused after.

Decisions and Trade-offs

Decision 1: Filtering for the same camera AND the same dance

- **Decision**: Currently, we start with the 1.3k dataset since this is for the MVP.
- **Rationale:** With this split, we won't need to implement any alignment since the camera and choreography length is fixed.
- **Trade-off:** We get significantly less data, and we would have to reconstruct the dataset when we want to scale up.

Decision 2: Choosing Mbps

- **Decision**: We went with 2 Mbps instead of 10 Mbps.
- **Rationale:** With the same duration. 10Mbps has a size of 514GB while 2Mbps has 82GB, for running efficiency and storage, we decided to go for the lower one.
- **Trade-off:** With lower Mbps we may get video with lower quality.

Code (initial version)

In feat/score-pipeline branch & merged in feat/a2 branch: A2/pipelines/score video data processing

Technologies Used

- Pandas– filtering for relevant data and constructing the meta data.
- Tqdm - showing download progress.
- FFmpeg – to cut the mp4 into smaller segments if needed.

Next steps

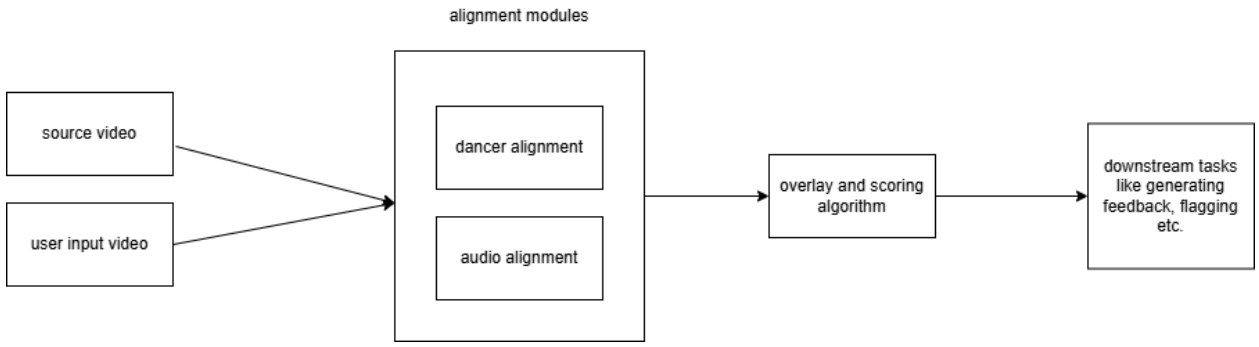
Currently, we only have 1.2k of high similarity score (this is manually randomly sampled and validated that the moves have a similar performance quality). For low score/ med score, we can implement later if needed. We also only have the 1.2k data locally; we are working on pushing it to the cloud.

Once the MVP is in place, we will also implement a script that parses the videos as inputs*, and gets the similarity scores. If the similarity scores are high, then we don't need to do anything; otherwise, we generate high similarity labels and fine-tune the algorithm model.
*We only have 3 folders with 3 dancers; we can simply use an if statement here to generate the combination. Otherwise, the folders all contain 2 dancers, no combination needed.

3.5 User Input Cleaning Pipeline

This is to be implemented after our MVP but we thought to mention this now.

Upon creation of our algorithm, the user will upload two videos: target and user dance video. These videos will go through the following pipeline:



Part Four: Infrastructure as Code Implementation

4.1 Main Objective

For part 4, we've implemented reproducible cloud infrastructure for our part 3 pipelines using infrastructure as code. The goal was to make deployment straightforward, maintainable, and environment-aware. The process of setting up the infrastructure is as easy as running a few command lines.

4.2 IaC framework and project structure

For the framework, we used *AWS CDK (TypeScript)*, which is an open-sourced IaC framework through AWS CloudFormation.

The infrastructure code location can be found at *A2/infrastructure/*.

- Entry point: *A2/infrastructure/bin/infrastructure.ts*.
- Stack definition: *A2/infrastructure/lib/infrastructure-stack.ts*.
- Infrastructure tests script: *A2/infrastructure/test/infrastructure.test.ts*.

We used CDK as its resource definitions are modular, typed, and version-controlled (and version control is one of major features we want as IaC).

Just like any other IaC, changes are previewable before deployment with "cdk diff" and "cdk synth". Environments are easily controllable by a single "STAGE" variable. Plus, we found it easy to use seamlessly with AWS.

For these reasons, we chose CDK.

Shell scripts note

You will see scripts/deploy_infra.sh and scripts/setup_aws_env.sh in the root.

We implemented infrastructure in AWS CDK as the source of truth. We only use shell scripts for operational convenience, like for credential refresh and standardized deploy commands, not for defining resources. For infra resource definition, we set out, you can check at *infrastructure-stack.ts*.

The infrastructure we set up and alignment with part 1-3

We provisioned the following resources:

- 4 S3 buckets
    - User upload bucket
    - Reference video bucket

    - Video validation bucket (used by video validation pipeline)
    - Audio validation bucket (used by audio validation pipeline)
- 2 DynamoDB
    - Users table
    - Sessions table

UsersTable and SessionsTable are just for saving user and session metadata.

Why did we add DynamoDB for part 4?

- S3 is our data plane for large binary artifacts (videos/audio/datasets and generated files).
- DynamoDB is our metadata plane which will act to support structured application records for users, sessions, score summaries, and links to S3 object keys.
- This keeps infrastructure aligned with our end product and supports future production features such as progress history and per-user session tracking, or whatever features we decide to expand on in the future.
- So DynamoDB is the forward-compatible infrastructure we added, but our A2 scope mainly focuses on validation pipelines with S3.

As an alignment with part 3 pipelines:

- Video pipeline (A2/pipelines/video data processing) stores/reads validation artifacts in the video validation S3 bucket.

- Audio pipeline (A2/pipelines/Audio data processing) stores source datasets under:
    - datasets/gtzan/
    - datasets/demand/ and stores generated outputs under configurable output prefixes (e.g., output/run1/).
    - gtzan is for clean music clips (target content to align) and demand is for real-world background noise recordings.

As an alignment with parts 1 and 2:

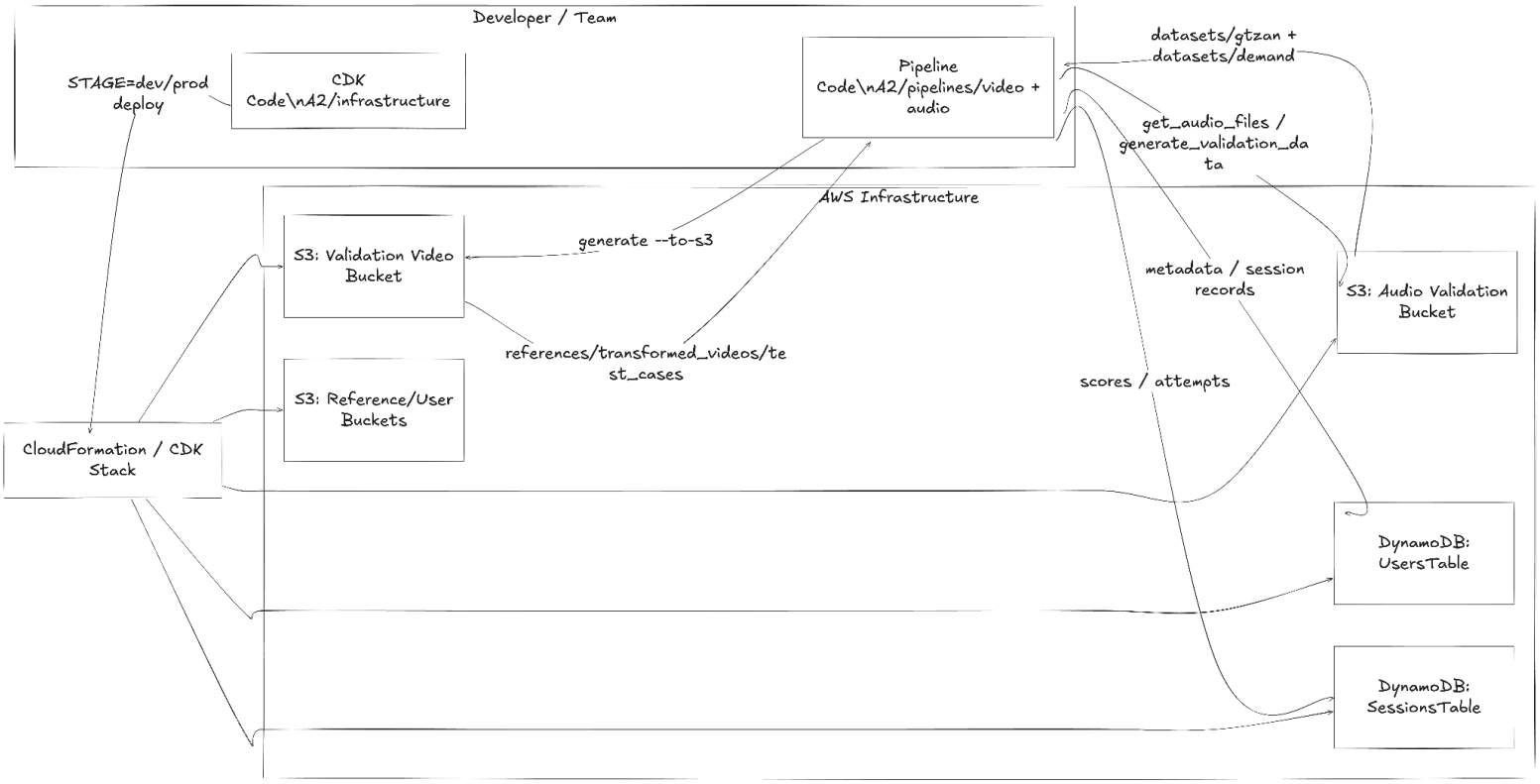
Our data pipeline in part 3 has taken into account of the aspirational datasets and the datasets we found as listed below:

- Video validation bucket
    - Part 1: we wanted the dataset in 1.1 and 1.4, also potentially in 1.5 as well once we have the MVP and move on to generating text responses.
    - Part 2:
        - What we used: this pipeline made use of AIST and AIST++ in 2.1. The ImpericalDance and FineDance are reserved as backup data as they contain useful 3D and other annotations. We decided to use a 2D implementation for now, and use these data to strengthen our model if

needed. We also created the Synthetic Scoring/Alignment Validation Dataset that lives or will live in this bucket.
        - To be used: the LLM-Synthesized Feedback Annotations in 2.4 will be used once we have the MVP and move on to generating text responses.
        -
- Audio validation bucket
    - Part 1: this calls for the datasets in 1.2, and the EBS pipeline called for the 1.3 dataset.
    - Part 2:
        - What we used: this pipeline made use of GZAN and DEMAND in 2.2.
        - To be used: we plan to use ADVSUC in 2.2 for the next step of the segmentation pipeline.
        -
- Scraped Reference–Cover Video Pairs (e.g., Instagram/TikTok “Dance Cover” Ecosystem) in 2.3 is reserved for testing and validation, and Scraped Step-by-step Tutorial Videos may be used by the EBS or deemed out of score as per 1.6.

This then concludes all sections and datasets in part 1 and part 2.

Diagram of our IaC + Part 3 Integration
The following diagram showcases an overview of our IaC + Part 3 integration:



4.3 Multiple Environments

We have implemented in a way that you can easily switch between “Dev” and “Prod” environments.

STAGE=dev represents the development environment where STAGE=prod represents the production version with stronger durability settings using retention behavior and PITR.

Our stack naming convention:

- TempoFlow-Infra-dev
- TempoFlow-Infra-prod

4.4 Deployment Workflow

We have a detailed step by step workflow with commands in the README file under the infrastructure folder. After refreshing the token with

AWS_ACCESS_KEY_ID=... AWS_SECRET_ACCESS_KEY=... AWS_SESSION_TOKEN=... ./scripts/setup_aws_env.sh

You can use following commands:

- cd /Users/yehyunlee/Documents/Repositories/TempoFlow/A2/infrastructure
- npm install
- npm run build
- npm run test
- STAGE=dev npx cdk synth > /tmp/tempoflow-dev.yaml
- STAGE=prod npx cdk synth > /tmp/tempoflow-prod.yaml
- STAGE=dev npx cdk deploy

Using the deploy shell script is another shortcut option too.

To verify deployed resource names:

- STACK=TempoFlow-Infra-dev
- VIDEO_BUCKET=$(aws cloudformation describe-stacks --stack-name "$STACK" --query "Stacks[0].Outputs[?OutputKey=='ValidationVideoBucketName'].OutputValue" --output text)
- AUDIO_BUCKET=$(aws cloudformation describe-stacks --stack-name "$STACK" --query "Stacks[0].Outputs[?OutputKey=='AudioValidationBucketName'].OutputValue" --output text)

After working on IaC, we have validated and ran with part 3’s video and audio data processing code.

4.5 Validation evidence from run logs

We've verified our processing script works fine with our infrastructure setup.

- CDK synth completed for both environments (synth-ok)
- Audio dataset verifier successfully found both datasets in S3 after upload:
    - datasets/gtzan/
    - datasets/demand
- Audio validation generation succeeded in AWS mode:
    - python generate_validation_data.py --mode aws --num-pairs 100 --s3-bucket "$AUDIO_BUCKET" --s3-output-prefix output/run1/ --aws-region "$AWS_DEFAULT_REGION"
- As noted, we have verified video processing work as well.

We have conducted IaC implementation, tested our data processing, and confirmed IaC resources were correctly provisioned and usable by pipeline code.

4.6 Maintainability

For the sake of team's maintainability and reproducibility, we have done the following:

- We have added two reusable shell scripts for the team to easily call our workflow steps with setup_aws_env.sh and deploy_infra.sh.

- In README.md file in the infrastructure folder, we have added some documentation, commands to run, lifecycle notes, for later reference, in case we forget about the details.

- We have added CDK tests to validate expected resource counts and production behaviours that's expected.

4.7 Known limitations and next improvements

We still have some limitations we would like to improve on.

1. Our AWS credential token setup workflow is currently annoying, where we need to type our tokens every 1 hour to refresh them. We could change to the SSO approach to make sessions live longer.

2. Current audio DEMAND upload in our run used a limited local subset. We can expand this for stronger noise diversity.

3. Video upload command currently prompts for credentials interactively. We could make this non-interactive profile-based auth for CI.

Part Five: Disaster Recovery Demonstration

5.1 What we simulated

We simulated a production disaster by deleting our production stack (TempoFlow-Infra-prod), then restoring it only through our IaC code (AWS CDK). No manual recreation through the AWS console was used for infrastructure resources.

5.2 What we verified in the demo

Data processing services / deployed application components:
We showed the deployed stack resources and validated that pipeline storage paths were accessible again after restore, including video validation and audio validation S3 locations.

1. **Database systems and their data**
   We demonstrated that DynamoDB tables (UsersTable and SessionsTable) were recreated and accessible after recovery. We also performed basic read/write checks to confirm the data plane worked post-restore.

2. **Configuration settings**
   We used stage-based deployment (`STAGE=prod`) and showed CloudFormation stack outputs before/after recovery to verify environment-level configuration consistency.

3. **Access controls and security settings**
   We verified AWS-authenticated access and validated security-related settings. For example, S3 public access blocking/encryption checks and DynamoDB continuous backup/PITR status where applicable.

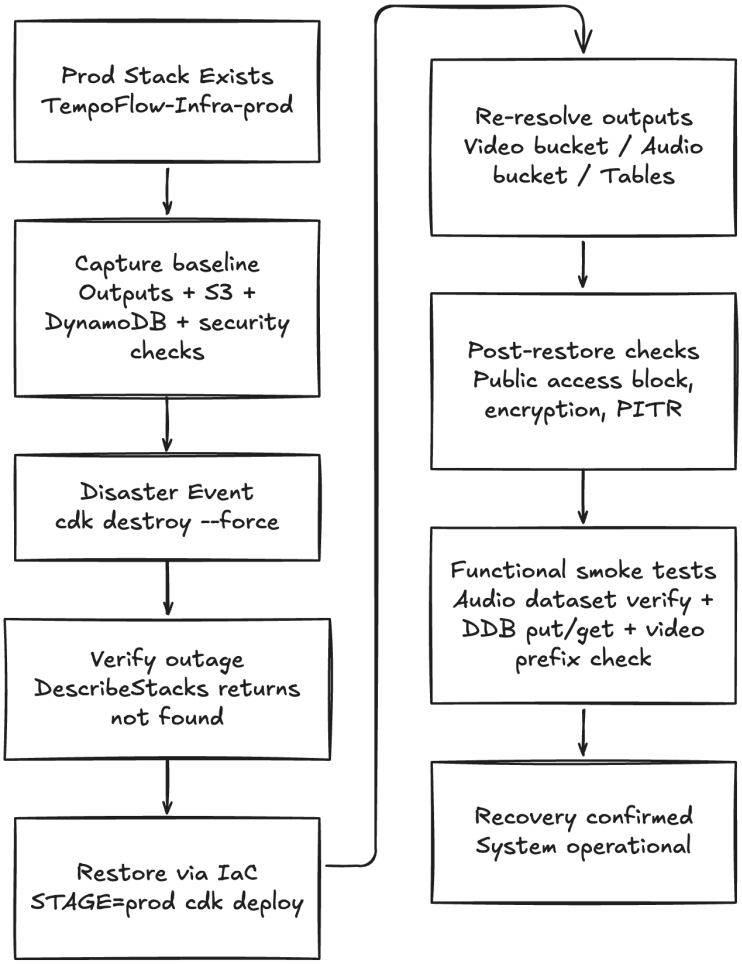
4. **Verification of system functionality**
   After the restore, we re-ran storage/pipeline smoke checks. For example, audio dataset verification commands against restored buckets and confirmed the system was operational again.

5.3 Demo Video

You can find our demo for part 5 in the following YouTube video:
https://youtu.be/zhJhDVGD_FY.
(Please do not share this video link publicly.)

If YT link is not clickable, use the following clip.

The demo showcases the following workflow in the diagram:



5.4 Disaster recovery result

As you can verify from our demo video, our infrastructure was successfully restored from code after deletion, and the core data/pipeline workflow returned to a functional state. This confirms that our infrastructure and the system are recoverable and reproducible through IaC rather than manual cloud setup.

Now, we no longer need to worry about future interns deleting our setup :)

Final Notes

Folder structure
You can ignore our “web-app” folder in the root directory. That’s our prototype we are currently playing with, and it is not fully supported, i.e., some code is still under a local branch uncommitted.

README files
Files and folders that are relevant for A2 are all located under “A2” folder. “pipelines” for part 3 and “infrastructure” for part 4. Each README file can be found under the video, audio, and score data processing folders. README for IaC can be found under the “infrastructure” folder.

Future exploration
Though we have so far only explored with YOLO, MediaPipe and other methods we have mentioned in our report, in addition to those and per Denys’ feedback, we are now taking a closer look into training, serving SAM 3 and transformer-based model Roboflow Rapid. We’ll have more to discuss about our experimentation in the next report.

User survey
We have created and sent out a “Dancer Feedback and Feature Desirability Study” survey to dancers to hear their thoughts, what they are looking for, struggles and feedback. It has 30 questions, and you can check it out here at https://forms.gle/uDMxFB9Lc5ydxXmo9.

We’ll reflect on those survey results to incorporate, improve our design, adjust scope, and improve our implementation for the next assignment.

If you consider yourself a dancer, feel free to fill it out!

Works Cited

Endo, K., Tsuchida, S., Fukusato, T., & Igarashi, T. (2024). *Automatic dance video segmentation for understanding choreography*. In *Proceedings of the 9th International Conference on Movement and Computing (MOCO '24)* (9 pp.). Association for Computing Machinery. https://doi.org/10.1145/3658852.3659076

Ionescu, C., Papava, D., Olaru, V., & Sminchisescu, C. (2014). Human3.6M: Large scale datasets and predictive methods for 3D human sensing in natural environments. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 36*(7), 1325–1339. https://doi.org/10.1109/TPAMI.2013.248

Li, R., Yang, S., Ross, D. A., & Kanazawa, A. (2021). *AI Choreographer: Music conditioned 3D dance generation with AIST++*. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 13401–13412). doi:10.1109/ICCV48922.2021.01315

Li, R., Zhao, J., Zhang, Y., Su, M., Ren, Z., Zhang, H., Tang, Y., & Li, X. (2023). *FineDance: A fine-grained choreography dataset for 3D full body dance generation*. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 10234–10243). doi:10.1109/ICCV51070.2023.00939

Okada, N., Iwamoto, N., Fukusato, T., & Morishima, S. (2015). *Dance motion segmentation method based on choreographic primitives*. In *Proceedings of the 10th International Conference on Computer Graphics Theory and Applications (GRAPP 2015)* (pp. 332–339). SCITEPRESS. https://doi.org/10.5220/0005304303320339

Thiemann, J., Ito, N., & Vincent, E. (2013). *The Diverse Environments Multi-channel Acoustic Noise Database (DEMAND): A database of multichannel environmental noise recordings*. *Proceedings of Meetings on Acoustics, 19*(1), 035081. https://doi.org/10.1121/1.4806631

Tsuchida, S., Fukayama, S., Hamasaki, M., & Goto, M. (2019). *AIST Dance Video Database: Multi-genre, multi-dancer, and multi-camera database for dance information processing*. In *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR 2019)* (pp. 501–510).

Tzanetakis, G., & Cook, P. (2002). *Musical genre classification of audio signals*. *IEEE Transactions on Speech and Audio Processing, 10*(5), 293–302. https://doi.org/10.1109/TSA.2002.800560

Zhong, Y., & Demiris, Y. (2024). *DanceMVP: Self-supervised learning for multi-task primitive-based dance performance assessment via transformer text prompting*. *Proceedings of the AAAI Conference on Artificial Intelligence, 38*(9), 10270–10278. doi:10.1609/aaai.v38i9.28893

Zhong, Y., Zhang, F., & Demiris, Y. (2023). *Contrastive self-supervised learning for automated multi-modal dance performance assessment*. In *ICASSP 2023 – 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 1–5). doi:10.1109/ICASSP49357.2023.10096824